Spam Base

Trabalho realizado por:
Beatriz Martins - 2023210538
Laura Barreto - 2023214375

Introdução

Este trabalho consiste na análise de um dataset que permite verificar, através de 57 características, se um email representa ou não spam. O objetivo é analisar um conjunto de dados com diferentes modelos, supervisionados ou não, e verificar o desempenho de cada um, analisando as suas métricas. De forma a facilitar a compreensão deste projeto, é recomendada a leitura do ficheiro "README.txt" que se encontra na pasta zip em conjunto com outros materiais.

Descrição dos dados

O dataset tem 4601 linhas e 58 colunas (57 características e uma coluna target) e, ao usar um histograma (figura 1), foi possível observar que não era balanceado, uma vez que tinha 2788 casos de emails que não eram spam e 1813 casos de emails que eram spam, como é possível observar na figura 2. Consequentemente, os modelos tendem a classificar melhor casos de emails que não são spam, possivelmente por terem contacto com um maior número de emails desse tipo. Portanto, é esperado que não sejam capazes de prever tão bem os casos de spam, o que constitui um problema, uma vez que os emails de spam são de maior necessidade de identificação por razões de segurança dos diferentes utilizadores. Assim, para continuar a nossa análise, decidimos utilizar dois métodos de pré-processamento de nomes, SMOTE e underSampling, que permitem balancear o dataset.

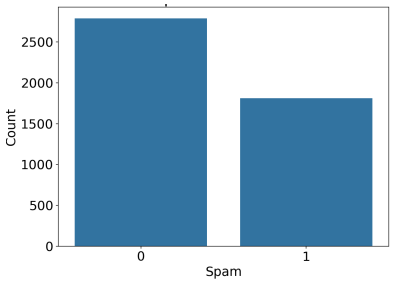

Figura 1 - Distribuição do dataset

```
spam
0    2788
1    1813
Name: count, dtype: int64
```

Figura 2 - Contagem dos casos do dataset

Pré Processamento com o SMOTE vs underSampling

Com o objetivo de equilibrar a distribuição dos exemplos de *spam* e *não spam*, foram aplicadas duas abordagens de pré-processamento, conforme foi sugerido pelo professor. No caso do SMOTE (*Synthetic Minority Over-sampling Technique*), são geradas novas amostras sintéticas para a classe com menor número de casos. O algoritmo KNN é utilizado de forma a interpolar entre os exemplos existentes, aumentando a quantidade de dados na classe em minoria (como é possível observar na figura 4). Em contraste, o underSampling reduz, aleatoriamente, o número de amostras da classe com maior número de casos, selecionando uma fração dos seus exemplos e retirando-os (como está ilustrado na figura 5). Ambas as técnicas têm como objetivo equilibrar a distribuição das classes no conjunto de treino, garantindo que o modelo seja treinado com dados mais balanceados. É importante destacar que estas abordagens são aplicadas exclusivamente aos dados de treino, com a finalidade de melhorar a capacidade do modelo de prever as duas classes com maior precisão e desempenho.

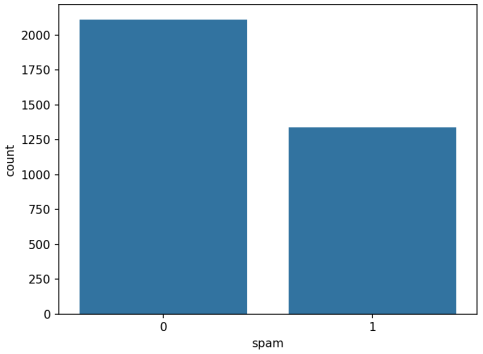

Figura 3 - Distribuição dos dados do conjunto de treino

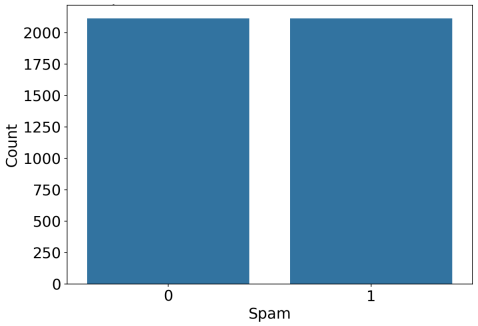

Figura 4 - Distribuição dos dados de treino após usar SMOTE

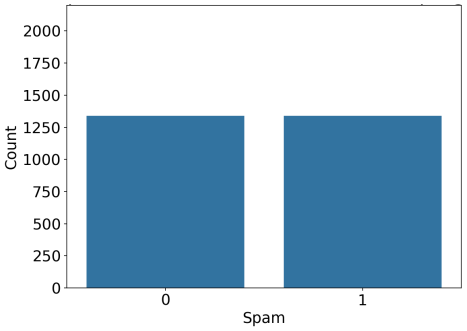

Figura 5 - Distribuição dos dados de treino após usar underSampling

Setup

Experiências para determinar o valor de maxdepth a ser usado no modelo da árvore de decisão:

No caso da árvore de decisão, faremos experiências para as três abordagens de pré-processamento, fazendo variar apenas o valor da profundidade máxima (maxdepth), de forma a descobrir que valores melhor se adequam, ou seja, que proporcionam melhores métricas. Decidimos fixar a entropia como critério por ser aquele que para nós era mais conhecido e mencionado. Fixamos também o parâmetro "random_state" a 42 (que permite obter os mesmos resultados, nas várias execuções de código).

Os valores que preenchem a tabela da figura 7 poderão ser encontrados, para cada uma das três abordagens, em ficheiros csv do tipo "tree_metrics_<nome da abordagem>.csv", presentes no ficheiro zip do projeto.

	Parâmetros			Métricas		
Cenário	maxdepth	criterion	random state	Accuracy	Recall	Precision
1	1	entropy	42			
2	2	entropy	42			
3	3	entropy	42			
4	4	entropy	42			
5	5	entropy	42			
6	6	entropy	42			
7	7	entropy	42			
8	8	entropy	42			
9	9	entropy	42			
10	10	entropy	42			

Figura 6 - Tabela dos cenários com diferentes valores maxdepth

Experiências para determinar o valor de k a ser usado no modelo KNN:

Para as três abordagens (sem balanceamento, pré-processamento com SMOTE e pré-processamento com underSampling), decidimos testar valores de k num intervalo de 1 a 15. O intuito desta experiência é encontrar o valor de k que permita maximizar as métricas de teste. Escolhemos testar o k neste intervalo, pois queríamos estudar o desempenho que o modelo tinha ao basear-se em mais ou menos exemplos para classificar um determinado email de forma correta. Após as 15 experiências para cada modelo, iremos obter tabelas do género da tabela da figura 6 com as métricas preenchidas, que se encontram, para cada uma das abordagens, em ficheiros csv de nome "knn_metrics_<nome da abordagem>.csv", que podem ser encontrados na pasta zip do projeto.

	Parâmetros	**Métricas**		
Cenários	Valores de k	Accuracy	Recall	Precision
1	1			
2	2			
3	3			
4	4			
5	5			
6	6			
7	7			
8	8			
9	9			
10	10			
11	11			
12	12			
13	13			
14	14			
15	15			

Figura 7 - Tabela dos cenários com diferentes valores de k

Os restantes modelos não passarão por este processo de experiências, por não terem parâmetros a serem testados.

K-means

O K-Means assume que os clusters são esféricos e bem separados, o que, por vezes, não acontece. Para este modelo, foi necessário utilizar o método PCA, de forma a resumir todas as variáveis presentes a apenas duas componentes, que foram usadas como eixos para os gráficos das figuras 8 e 9. Foi também necessário normalizar os dados, visto que, este modelo usa a distância euclidiana para medir as distâncias entre os pontos. Os resultados mostram que o método não conseguiu separar bem as classes de spam e não spam, sugerindo que os dados podem não ser naturalmente agrupáveis através de um modelo baseado em clusters. Isto indica que outros métodos como modelos supervisionados (como por exemplo, Naive Bayes e Árvores de Decisão) poderão ser mais adequados para este tipo de problema.

Assim, é desnecessário usar o método não supervisionado DBSCAN, visto que, sendo um método que agrupa clusters consoante a densidade de pontos numa dada região, iria agrupar a área mais densa (visível na figura 9), num único cluster, o que não corresponderia à realidade. Para além disso, classificaria como outliers os pontos mais distantes do cluster, ou seja, que não seriam atingíveis a partir dele. Portanto, a divisão através deste método não corresponderia a uma divisão precisa de casos de spam e não spam, daí termos decidido não utilizar este método.

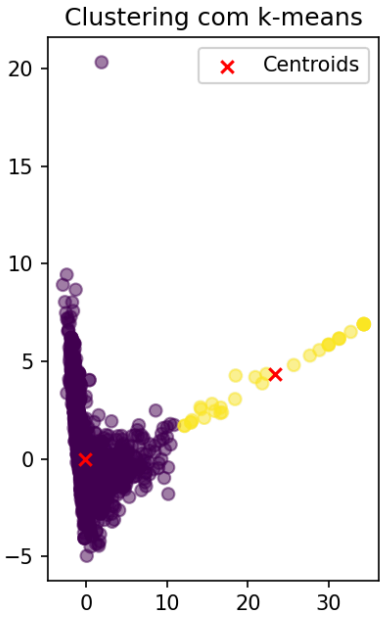

Figura 8 - Clusters formados após k-means

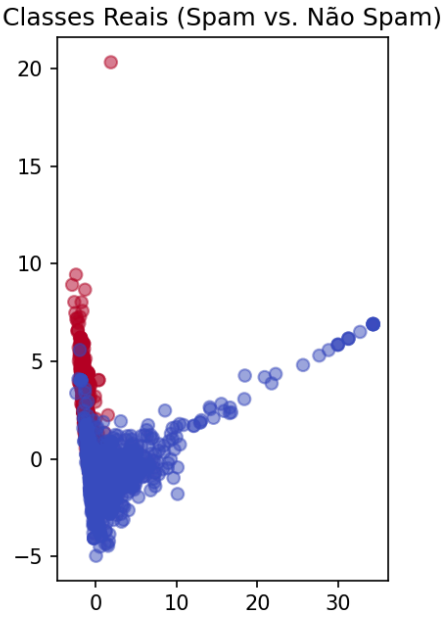

Figura 9 - Separação dos dados nas classes sem aplicar k-means

Árvore de decisão

Este modelo consiste numa árvore com caminhos que permitem classificar novos emails. Esses caminhos começam na raiz (com entropia máxima de 1) e terminam em qualquer folha (com entropia zero).

Experiências para determinar o parâmetro maxdepth na árvore de decisão:

Ao procedermos às experiências para determinar os melhores valores de profundidade para a árvore de decisão, para cada uma das três abordagens (dados sem balanceamento, SMOTE e underSampling), obtivemos as métricas de teste apresentadas nos gráficos das figuras 10, 11 e 12.

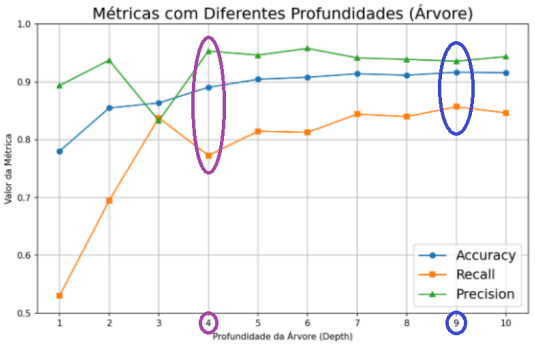

Figura 10 - Métricas do modelo da árvore de decisão sem balanceamento dos dados

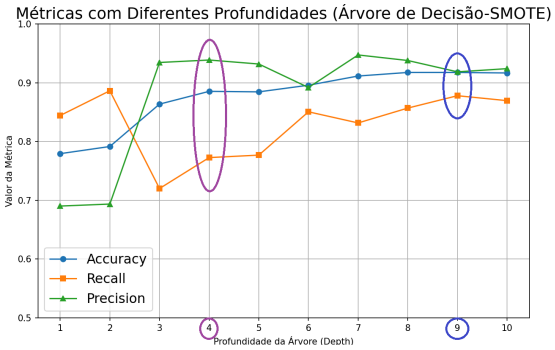

Figura 11 - Métricas do modelo da árvore de decisão com SMOTE

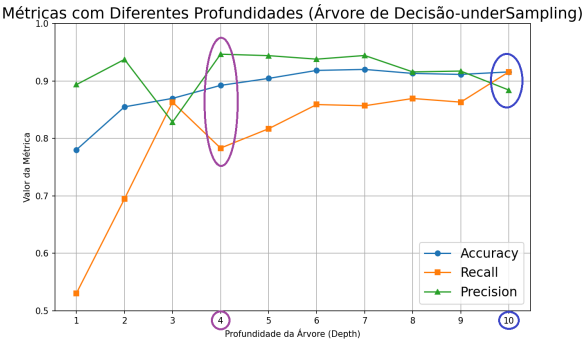

Figura 12 - Métricas do modelo da árvore de decisão com underSampling

Através da análise dos gráficos das figuras 10, 11 e 12, podemos entender que as profundidades que resultam em métricas mais equilibradas e razoáveis, para os três pré-processamentos (nenhum balanceamento, SMOTE e underSampling), são 9, 9 e 10, respetivamente (como se pode observar nos círculos azuis dos gráficos acima). É importante destacar que para as três abordagens, de uma forma geral, temos valores de recall mais baixos comparados aos valores de precisão, ou seja, o modelo obtém mais falsos negativos do que falsos positivos. Desta forma, é mais fácil para o modelo detetar os emails que não são spam comparativamente aos que o são, mesmo que este tenha passado por um treino com dados balanceados destas duas classes.

Contudo, iremos usar maxdepth com o valor 4 para as três abordagens, visto que, facilitará a visualização da árvore, permitirá a comparação dos três pré-processamentos nas mesmas condições e impedirá a ocorrência de overfitting.

Podemos prever de imediato que as métricas nas três abordagens são bastante semelhantes, como se pode observar nos círculos roxos dos gráficos das figuras 10, 11 e 12.

Árvores de decisão com maxdepth 4 nas três abordagens:

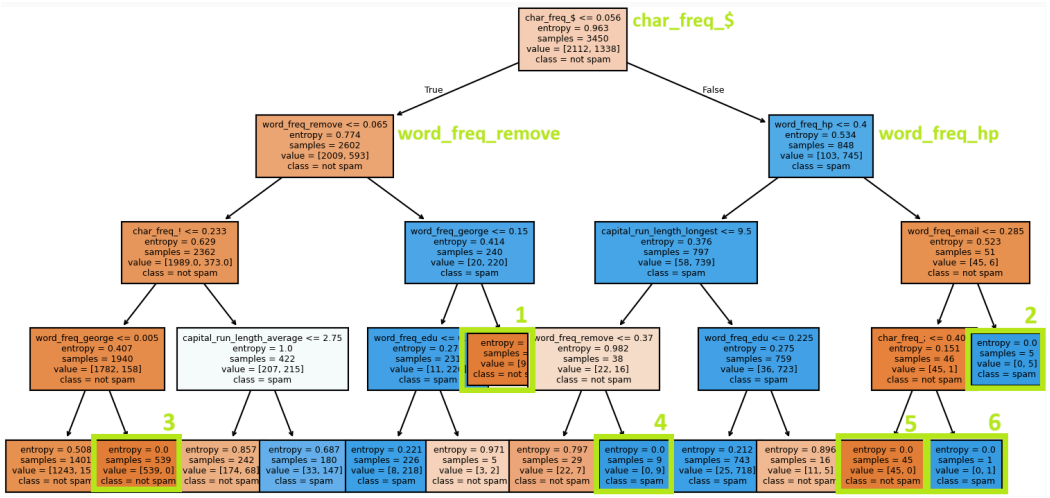

Figura 13 - Árvore de decisão sem nenhum método de balanceamento com as folhas a verde

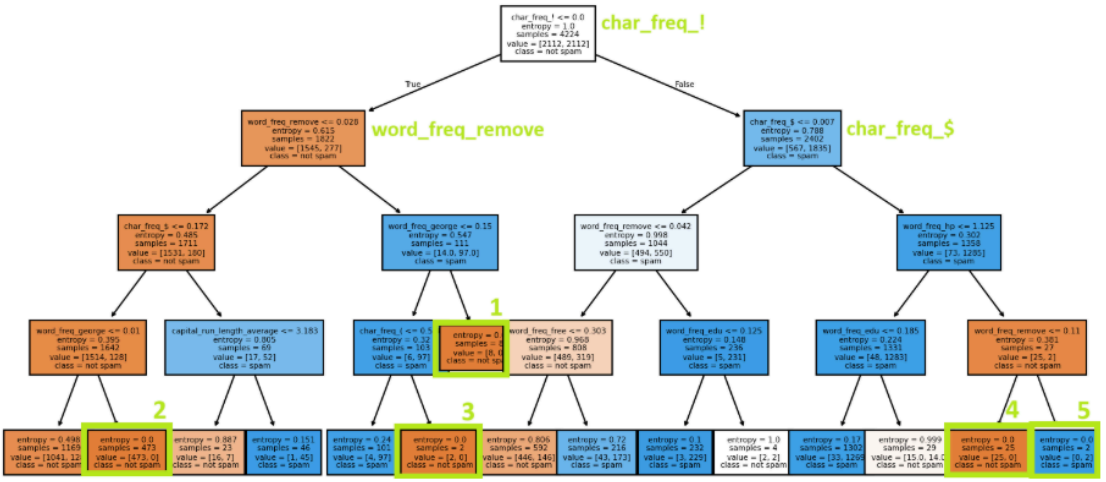

Figura 14 - Árvore de decisão com método SMOTE com as folhas a verde

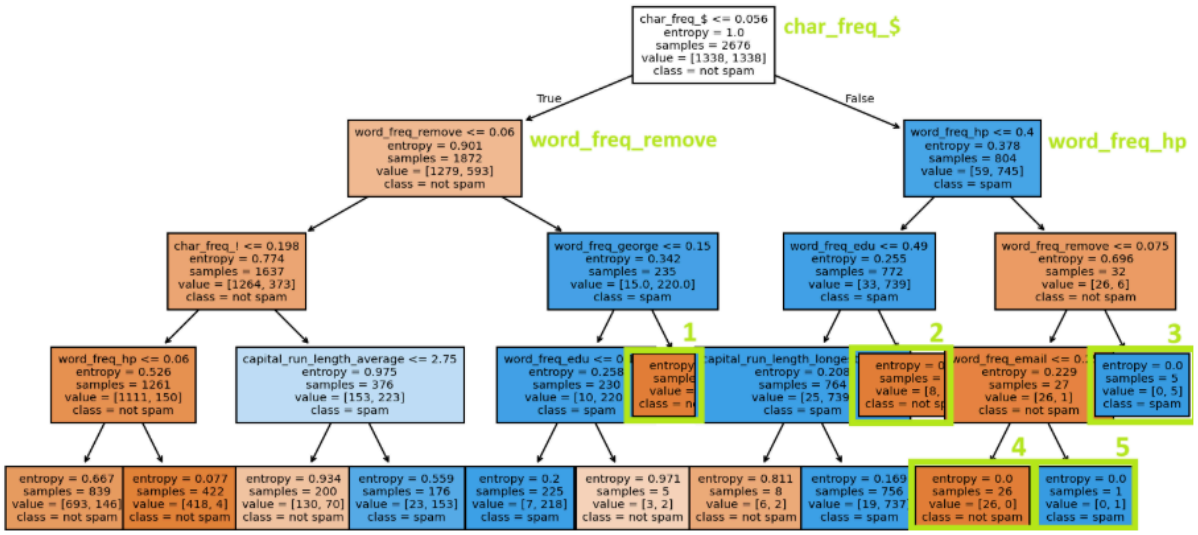

Figura 15 - Árvore de decisão com método undersampling com as folhas a verde

Podemos analisar que as três abordagens obtiveram uma árvore de decisão bastante semelhante e consideraram como raízes as frequências dos caracteres “!” e “$”, como é possível notar a verde nos gráficos acima. A frequência da palavra “remove” e “hp” também é uma característica indicativa de spam, como está representado a verde no segundo nível das árvores de decisão das figuras 13 e 15.

Comparando os resultados das árvores com e sem balanceamento, podemos concluir que as duas árvores com dados balanceados têm cinco folhas ao longo dos seus 4 níveis de profundidade, enquanto que a árvore que não passou por nenhum tipo de pré-processamento, obteve seis folhas ao longo dos seus 4 níveis, como se pode observar nos quadrados verdes das figuras 13, 14 e 15.

Nas figuras abaixo, estão representados os resultados do desempenho deste modelo através de matrizes de confusão e uma tabela com métricas como accuracy, precision e recall.

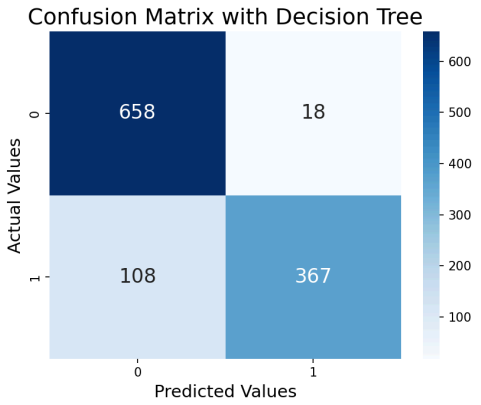

Figura 16 - Matriz de confusão da árvore de decisão caso sem balanceamento dos dados

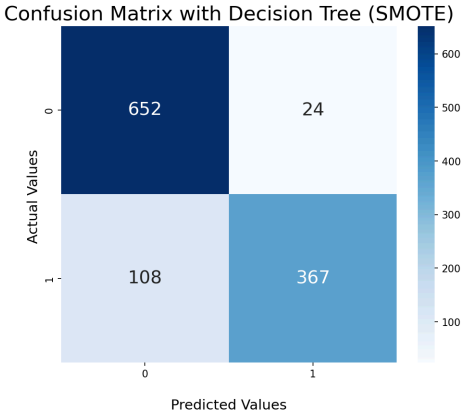

Figura 17 - Matriz de confusão da árvore de decisão no caso de utilizar o método SMOTE

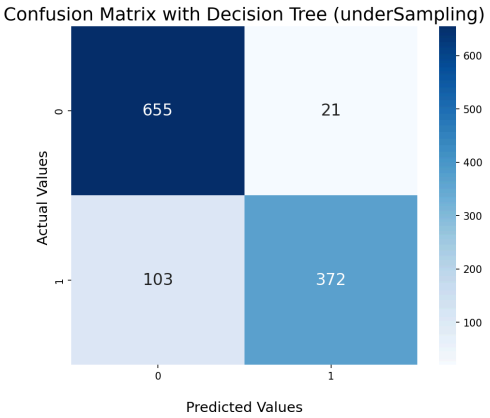

Figura 18 - Matriz de confusão da árvore no caso de utilizar método de undersampling

	Accuracy (%)	Recall (%)	Precision (%)	Tempo de treino (s)
Sem balanceamento	89.05	77.26	95.32	0.0484
SMOTE	88.53	77.26	93.86	0.0983
underSampling	89.23	78.32	94.66	0.0517

Figura 19 - Tabela com resultados das métricas no conjunto de teste para as 3 abordagens

Pelas matrizes de confusão, conseguimos visualizar que este modelo realmente conseguiu detetar mais facilmente os casos de não spam comparativamente aos de spam. Podemos também notar que o modelo tem mais facilidade em evitar falsos positivos do que a identificar corretamente os casos de spam, pois os valores de precision são superiores aos valores de recall, como é visível na fig 19. Tudo isto significa que embora o modelo possa ser cuidadoso ao considerar casos como spam, tem tendência para deixar passar alguns emails de spam não detetados, gerando mais falsos negativos.

Já no caso das métricas, elas são bastante idênticas nas três abordagens, daí podermos concluir que, para este modelo, o uso de técnicas de balanceamento não trouxe diferenças significativas. Posto isto, este modelo será comparado com outros, sem passar por nenhum tipo de balanceamento.

KNN

O modelo K-Nearest Neighbors prevê a classe de um e-mail, calculando a distância euclidiana desse email a todos os outros e escolhendo apenas os k mais próximos para o classificar. Antes de inicializar este modelo, é necessário passar por um processo de normalização dos dados de forma a que todas as variáveis contribuam de igual forma para este modelo.

Experiências para determinar o parâmetro k no KNN:

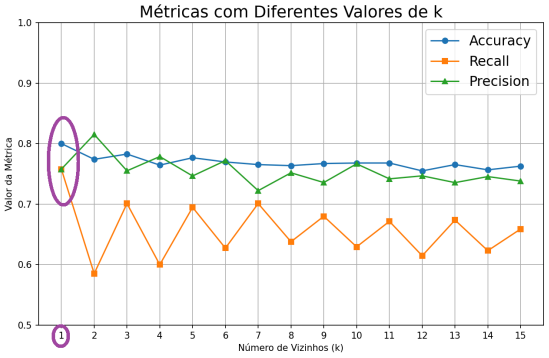

Figura 20 - Métricas do modelo consoante os valores de k nos dados não balanceados

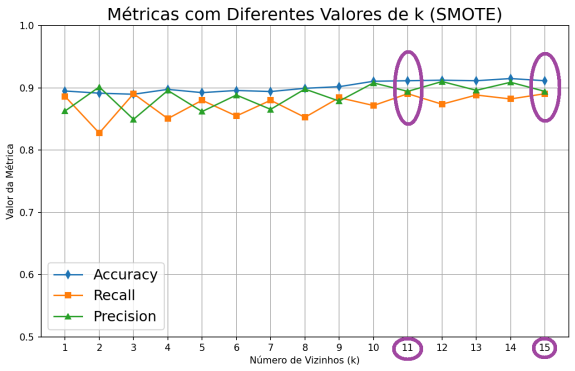

Figura 21 - Métricas do modelo consoante os valores de k nos dados usando SMOTE

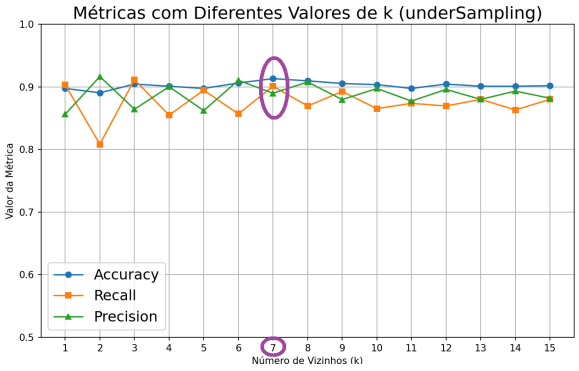

Figura 22 - Métricas do modelo consoante os valores de k usando underSampling

No gráfico da figura 20, os dados não se encontram balanceados, e é possível verificar que o valor de k que traz melhores resultados é 1. Usando k = 1, conseguimos melhores valores tanto de accuracy como de recall e ainda um valor de precisão razoável. O facto de um k tão pequeno dar melhores resultados, pode estar relacionado com a estrutura do nosso dataset, uma vez que não consiste em duas classes bem separáveis e agrupáveis, como vimos no método de clustering.

No caso do gráfico da figura 21 em que os dados são pré-processados usando SMOTE, obtemos as melhores métricas tanto com k igual a 15 como com k igual a 11, sendo os valores das métricas exatamente iguais para ambos (visível no ficheiro "knn_metrics_SMOTE.csv"). Tendo em conta que o modelo é computacionalmente mais rápido usando um valor de k menor, escolhemos o k com valor 11. Além disso, estes valores mais elevados de k podem estar relacionados com o facto de o método SMOTE gerar novos pontos sintéticos, o que pode exigir um maior número de vizinhos de forma a garantir uma classificação mais estável e representativa.

Por último, com o uso de underSampling, o melhor valor de k será 7 e, pela primeira vez, verificamos um valor de recall ligeiramente superior comparativamente ao valor da precision. É importante notar que, de uma forma geral, usando abordagens de balanceamento, obtemos métricas muito melhores, comparadas às métricas dos dados não balanceados. Assim, é possível concluir que foi uma mais valia o uso de modelos de balanceamento para este modelo.

Nas figuras abaixo (23, 24 e 25), estão representados os resultados do desempenho deste modelo através de matrizes de confusão e ainda uma tabela com métricas como accuracy, precision e recall.

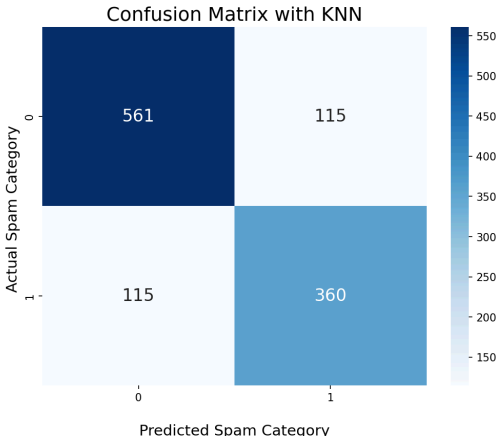

Figura 23 - Matriz de confusão de KNN sem balanceamento dos dados

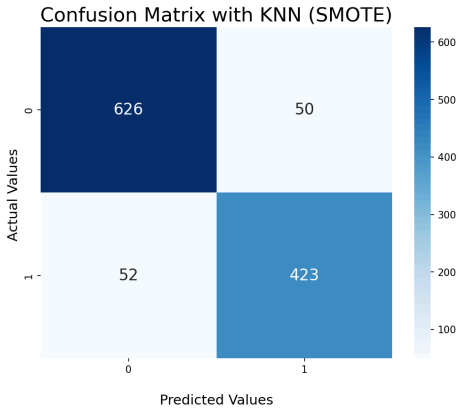

Figura 24 - Matriz de confusão de KNN no caso de utilizar o método SMOTE

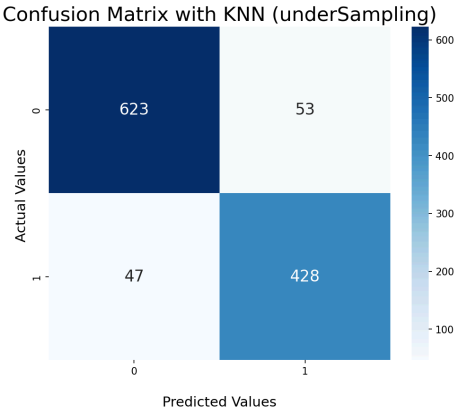

Figura 25 - Matriz de confusão de KNN no caso de utilizar o método undersampling

	Accuracy	Recall	Precision	Tempo de treino
Sem balanceamento	0.8002	0.7579	0.7579	0.0125
SMOTE	0.9114	0.8905	0.8943	0.0033
underSampling	0.9131	0.9011	0.8898	0.0232

Figura 26 - Tabela com resultados das métricas do conjunto de teste para as três abordagens

Observando os resultados, podemos notar que, ao contrário do modelo anterior, não tivemos tantos falsos negativos e conseguimos obter métricas mais equilibradas e razoáveis, como se pode visualizar nas figuras 23, 24 e 25, para cada tipo de pré-processamento. É, também, bastante percetível que qualquer métrica com os dados balanceados obteve valores superiores comparativamente aos valores das métricas com os dados não balanceados. Isto quer dizer, que o balanceamento dos dados para este modelo foi bastante benéfico.

No final, escolhemos o pré processamento com underSampling, visto que este consegue identificar mais facilmente os emails de spam, possuindo os valores de recall e accuracy mais elevados.

Assim, a nossa escolha parte do princípio que é mais importante detetar quando o email é spam, visto que este pode possuir algum tipo de malware. Desta forma, mesmo que corramos o risco de avaliar incorretamente um email que não era spam e eventualmente impedir o utilizador de ter acesso à sua mensagem, acaba por ser preferível, uma vez que os emails de spam são mais facilmente identificados o que diminui o perigo para esse utilizador.

Regressão Logística

A regressão logística é um modelo muito semelhante à regressão linear, no entanto, em vez de prever um valor contínuo, prevê uma probabilidade associada a uma classe. Este modelo usa a função sigmoide para transformar a saída numa escala compreendida entre 0 e 1, tornando-a adequada para problemas de classificação. Para inicializar este modelo, também procedemos à normalização das características, para que estas contribuíssem de igual forma para o modelo.

Nas figuras abaixo (27, 28 e 29), estão representados os resultados do desempenho deste modelo.

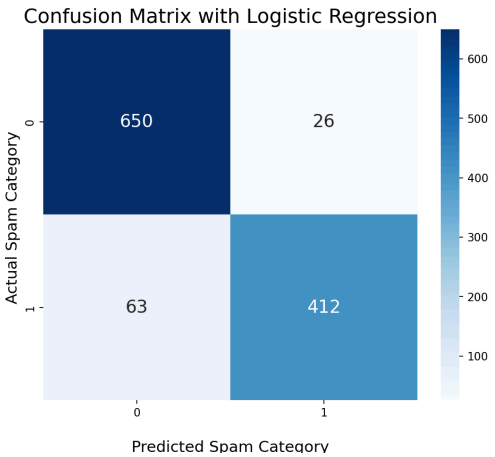

Figura 27 - Matriz de confusão da regressão logística sem balanceamento

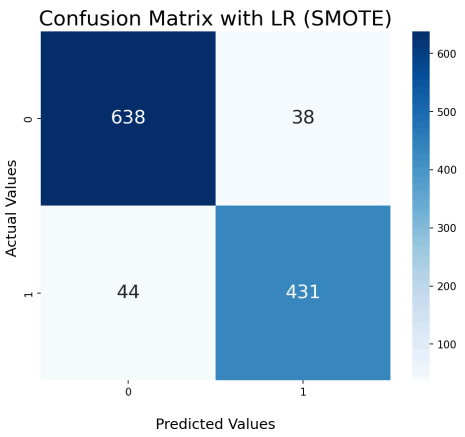

Figura 28 - Matriz de confusão da regressão logística utilizando SMOTE

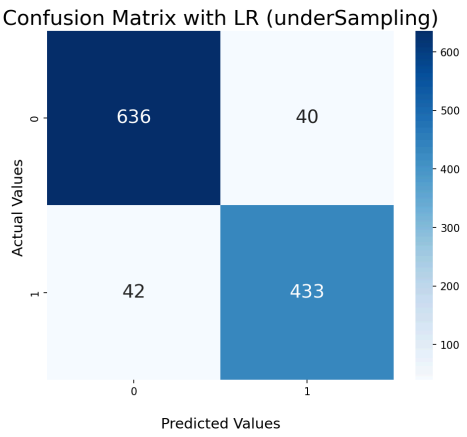

Figura 29 - Matriz de confusão da regressão logística utilizando undersampling

	Accuracy (%)	Recall (%)	Precision (%)	Tempo de treino (s)
Sem balanceamento	92.27	86.74	94.06	0.0476
SMOTE	92.88	90.74	91.90	0.0765
underSampling	92.88	91.16	91.54	0.0460

Figura 30 - Tabela com resultados das métricas do conjunto de teste para as três abordagens

Ao analisar a figura 30, é possível concluir que o modelo teve um excelente desempenho, uma vez que os valores das suas métricas foram todos acima de 90%, com exceção do valor recall nos dados não balanceados. Comparando as suas métricas podemos concluir que com balanceamento, obtivemos melhores resultados, a nível da métrica recall e accuracy. No entanto, a nível de precision, o conjunto sem pré-processamento obteve um valor ligeiramente mais elevado comparativamente às outras abordagens.

Desta forma, este modelo sobressai quando os dados passam por pré-processamento com undersampling, uma vez que, o valor de recall neste método foi superior, o de accuracy igual e o de precision apenas ligeiramente menor quando comparado aos valores de SMOTE.

Naive Bayes

O modelo Naive Bayes tenta prever a classe dos exemplos no teste, tendo em conta todos os exemplos de treino. Este é um modelo que por norma consegue obter bons resultados, no entanto, isso não se verificou no nosso caso, como é possível observar nas figuras seguintes.

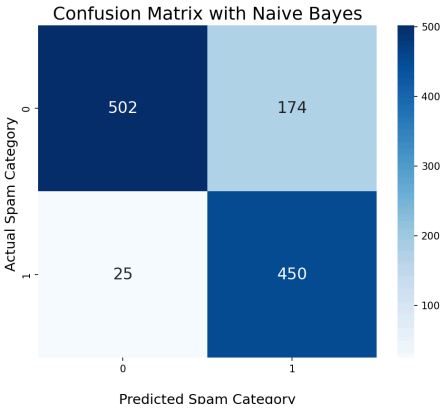

Figura 31 - Matriz de confusão do Naive Bayes sem balanceamento

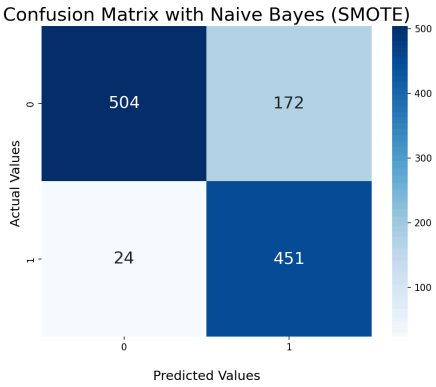

Figura 32 - Matriz de confusão da Naive Bayes utilizando SMOTE

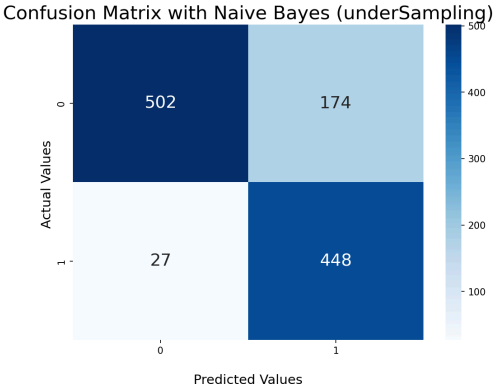

Figura 33 - Matriz de confusão da da Naive Bayes utilizando undersampling

	Accuracy (%)	Recall (%)	Precision (%)	Tempo de treino (s)
Sem balanceamento	82.71	94.74	72.12	0.0089
SMOTE	82.97	94.95	72.39	0.0085
underSampling	82.56	94.32	72.03	0.0176

Figura 34 - Tabela com resultados das métricas do conjunto de teste para as três abordagens

Analisando as figuras 31, 32 e 33, podemos observar bastantes falsos positivos, o que quer dizer que, em alguns emails, classificamo-los como spam, quando na verdade não o são.

Observando a tabela das métricas (figura 34), podemos perceber que temos valores de recall bastante elevados, contudo são à custa de valores de precisão mais baixos, o que não é desejável. Isto porque estamos a prever quase todos os emails spam corretamente, à custa de um número elevado de falsos positivos. Desta forma, o utilizador poderá não ler emails com informações relevantes por este modelo o ter considerado spam.

Ao comparar os resultados nas três abordagens, concluímos ainda que todas as métricas tinham valores ligeiramente superiores no caso dos dados que passaram por pré-processamento com SMOTE, sendo, desta forma, este método que escolheríamos para processar os dados antes de usar este modelo.

Comparação de resultados e discussão:

Escolhendo o pré-processamento que obteve melhores resultados em cada um dos modelos obtemos a tabela abaixo:

Tipo de pré processamento	**MODELO**	**Accuracy (%)**	**Recall (%)**	**Precision (%)**	**Tempo de treino (s)**
sem balanceamento	**Decision Tree**	80.02	75.79	75.79	0.0125
underSampling	**KNN**	91.31	90.11	88.98	0.0232
underSampling	**Logistic Regression**	92.88	91.16	91.54	0.0460
SMOTE	**Naive Bayes**	82.97	94.95	72.39	0.0085

Figura 35 - Tabela de comparação dos modelos com as correspondentes abordagens escolhidas

-> valores superiores a 90%
-> valores entre 80% e 90%
-> valores entre 70% e 80%

À primeira vista, o modelo com menor performance, ou seja, que não seria usado para o dataset em questão, seria a árvore de decisão, por ter os valores mais baixos em praticamente todas as métricas. Mesmo não sendo o modelo mais indicado para este caso, permitiu-nos entender que a presença de caracteres e palavras como "!", "$", "remove" e "hp", são indicativos de spam, dando-nos a conhecer melhor os critérios característicos de um email deste tipo. No entanto, é importante destacar que este desempenho mais baixo pode ter sido causado pelo parâmetro de profundidade máxima (maxdepth), que foi ajustado a 4 para uma visualização mais facilitada, mas que podia impedir a árvore de capturar totalmente a complexidade dos dados, visto ser um valor mais reduzido.

O segundo modelo com piores resultados foi o Naive Bayes, com métricas, em geral, com valores superiores comparativamente ao modelo anterior, mas com uma menor precision, também não seria a melhor opção para este dataset, uma vez que mesmo conseguindo classificar corretamente uma grande quantidade de casos de spam (valor de recall elevado), acaba por fazer isso à custa de um valor inferior de precision, levando a que o modelo classifique uma quantidade considerável de emails autênticos como spam. Isto seria prejudicial para o utilizador, que iria perder informações que para ele poderiam ser importantes. Uma razão que pode ter contribuído para este mau desempenho, poderá estar relacionada com o facto do modelo considerar as variáveis como sendo independentes. No entanto, isso não acontece no nosso dataset, levando à conclusão de que o que o modelo assume, está errado. Tudo isto pode ser provado através da figura 35, em que é percetível a existência (principalmente mais visível no centro) de variáveis bastante correlacionadas.

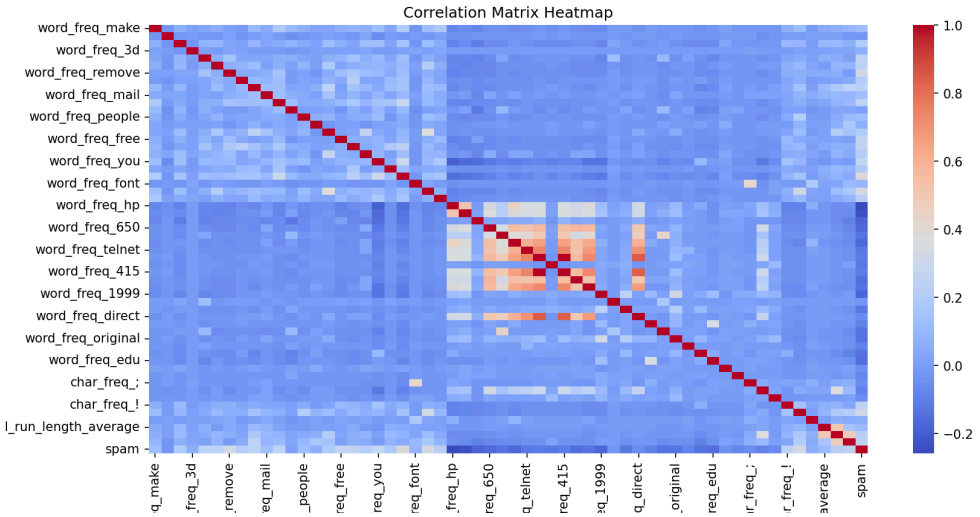

Figura 36 - Mapa das correlações entre todas as características do dataset

O modelo KNN foi o segundo com melhor desempenho, tendo obtido resultados muito favoráveis das métricas em geral. Verifica-se um valor de recall ligeiramente superior comparado ao da precision, o que indica que o modelo tem uma maior capacidade de identificar corretamente os exemplos positivos de spam. Adicionalmente, a etapa experimental foi crucial para este modelo, pois permitiu identificar que o valor de k = 7 era o mais adequado, maximizando assim as métricas e garantindo um desempenho mais promissor. Este modelo seria uma boa opção para o nosso dataset, no entanto ainda existe uma alternativa melhor, em que todas as métricas têm valores superiores às deste modelo, como veremos a seguir.

O modelo que teve melhor desempenho, obtendo os valores mais elevados em todas as métricas foi a regressão logística. Estes valores foram todos superiores a 90%, o que mostra que a regressão logística consegue classificar as duas classes de forma correta, equilibrada e precisa. Uma razão para o facto de este modelo ter obtido tão bons resultados com este dataset, poderá ter a ver com as características do mesmo. Isto porque, se os atributos tiverem uma relação clara e definida com a coluna target (alvo), as classes são mais facilmente separadas por uma fronteira linear, o que aumenta o desempenho da regressão logística.

Desta forma, este modelo seria a melhor opção a utilizar, uma vez que não só teria sensibilidade ao classificar emails que não são spam, de forma a não impedir os utilizadores de lerem mensagens importantes que foram classificadas como spam, mas também prestaria alta atenção às características indicativas de spam de forma a classificar os emails deste tipo corretamente, para não constituírem um perigo para o utilizador.

Comparando agora os métodos de pré-processamento, o underSampling trouxe melhores resultados em dois dos quatro modelos. Isto pode acontecer devido à sua redução do número de amostras na classe com mais exemplos, o que facilita o ajuste dos dados ao modelo em específico, treinando-o com igual número de amostras tanto de spam como de não spam. No entanto, é importante destacar que um método deste género poderia ser prejudicial se tivéssemos um dataset mais pequeno, uma vez que perderia muitas informações valiosas.

Conclusão

Com este trabalho, verificámos que o uso de técnicas de pré-processamento foi bastante positivo principalmente nos modelos KNN, e regressão logística, mas também revelou bons resultados quando aplicada no Naive Bayes. Contudo esta técnica mostrou-se desnecessária no modelo da árvore de decisão, por não ter obtido métricas melhores comparativamente à abordagem não balanceada.

Assim, entendemos muito melhor os modelos que podem ser utilizados para classificar conjuntos de dados de diferentes formas. Para além disso, ficamos a compreender melhor as métricas utilizadas para os avaliar, a compará-los através dessas métricas e, ainda, a identificar o modelo que mais se adequa a uma situação em específico.

Desafios

Ao longo deste projeto, surgiram algumas dificuldades, entre elas no enquadramento da árvore de decisão, na escolha de um limite máximo de profundidade e ainda de um tamanho adequado de fonte para as palavras que se encontravam nos gráficos. Para além disso, a técnica smote e underSampling trouxeram algumas dúvidas, pois não tínhamos a certeza de como seriam usadas e, só após alguma pesquisa, entendemos como funcionavam. Ao analisar o gráfico da correlação, também tivemos algumas dificuldades, devido ao número elevado de características do dataset. Além disso, pretendíamos também analisar o tempo de treino de cada modelo. No entanto, apercebemo-nos de que este não se mantinha constante em cada execução, uma vez que é influenciado por vários fatores, como a quantidade de processos em execução no CPU. A ideia inicial era utilizar um boxplot do tempo de treino ao longo de 30 execuções, mas, por falta de tempo, não conseguimos implementar essa abordagem na totalidade. Por último, ao analisarmos o guião do projeto, surgiram algumas incertezas sobre quais informações deveriam constar no relatório, bem como sobre o tipo de relatório e materiais a entregar.